



VISUALISATION TRIPTERON ROBOT

นนทพงศ์ เสถียรพงศ์, 66340500025
วรภัทร ภัทรเปรมเจริญ, 66340500049

บทคัดย่อ

โปรเจกต์นี้มีวัตถุประสงค์เพื่อศึกษาการเคลื่อนที่ของหุ่นยนต์ Tripteron ประเภท Parallel Manipulator (PM) ซึ่งเป็นหุ่นยนต์ที่สามารถเคลื่อนที่เชิงเส้นได้ทั้ง 3 แกน เพื่อควบคุมให้ปลายมือจับ (End Effector) ไปยังตำแหน่งที่ต้องการภายในพื้นที่ทำงานแบบสามมิติ โดยใช้การขับเคลื่อนจากมอเตอร์จำนวน 3 ตัว ที่ติดตั้งบนแขนกลแต่ละข้าง การศึกษาจะมุ่งเน้นไปที่การคำนวณทางจลนศาสตร์ของระบบ Forward Kinematic เพื่อแปลงข้อมูลจาก Joint Space ไปสู่ Task Space ผ่านการกำหนด Denavit-Hartenberg (DH) Parameter ผลลัพธ์ที่ได้จะประกอบด้วยข้อมูลตำแหน่งเชิงเส้น (Translation) และการหมุนเชิงมุม (Rotation) ของปลาย End Effector และ inverse kinematic จาก Task Space ไป joint Space และคำนวณหา different kinematic ของตัวหุ่นเพื่อหา Jacobian นำมาใช้หาความเร็วของปลายมือจับกับข้อต่อ และ static force ภายในระบบที่ทำให้ joint หมุน จากนั้นจะนำไปทำ Simulate ผ่าน โปรแกรม MATLAB เพื่อทำการตรวจสอบความถูกต้องและสามารถใช้ในการ Simulate เมื่อ Configuration เปลี่ยน

คำสำคัญ: DH Parameter, Task Space, Configuration, Forward kinematic, Inverse kinematic, Jacobian, static force

1. บทนำ (Introduction)

1.1 จุดประสงค์โครงการ

1. เพื่อพัฒนาและควบคุมหุ่นยนต์ Tripteron แบบ Parallel Manipulator ที่สามารถเคลื่อนที่ได้ 3 แกนในพื้นที่ทำงานเชิงสามมิติ
2. เพื่อศึกษาวิธีการคำนวณทางจลนศาสตร์ของระบบ Forward Kinematic โดยอาศัย Denavit-Hartenberg (DH) Parameter
3. เพื่อแปลงตำแหน่งจาก Joint Space ไปยัง Task space และหาตำแหน่งของ End effector
4. เพื่อคำนวณหาอัตราการเปลี่ยนของ Joint ในรูปแบบของอัตราการเปลี่ยนแปลงความเร็ว และ Torque
5. เพื่อทำระบบ SIMULATE ผ่าน MATLAB เพื่อทำการตรวจสอบโดยระบบคอมพิวเตอร์

1.2 ขอบเขต

- 1) ศึกษาและพัฒนาหุ่นยนต์ Tripteron Robot ชนิด Parallel Manipulator ที่มีอิสระ 3 แกนเชิงเส้นเท่านั้น (3-DOF Translational)
- 2) ใช้ตัวแบบคำนวณทางจลนศาสตร์เชิงตรงและเชิงกลับสำหรับระบบ Forward Kinematic โดยกำหนดพารามิเตอร์ตาม Denavit-Hardenberg (DH Parameter)
- 3) การคำนวณและแสดงผลจะครอบคลุมเฉพาะตำแหน่ง (Translation) ของ End Effector ใน Task Space

- 4) ขอบเขตของระบบควบคุมอยู่ในระดับการเคลื่อนที่พื้นฐาน โดยไม่รวมถึงการควบคุมแรง, การหลบหลีกสิ่งกีดขวาง หรือการปรับแก้ความคลาดเคลื่อนเชิงโครงสร้างขั้นสูง
- 5) การทดลองประเมินผลจะเน้นความถูกต้องของตำแหน่งปลาย End Effector ภายใต้สภาวะการทำงานปกติ
- 6) โปรแกรมที่จะใช้คำนวณจะมาจาก MATLAB เพียงเท่านั้น
 - a) 1.จะไม่มีการจำลองโมเดลมอเตอร์ (trajectory)ออกมาจะจำลองแค่การเคลื่อนที่ให้ถึงจุดเป้าหมายตามเวลาเท่านั้น
 - b) 2.end-effector ที่จะทำให้จะเป็นการจำลองให้ฐานเป็น 4 จัตุรัสเพียงเท่านั้น
 - c) 3.จะยังไม่มีการคำนวณของโมเดลจริง ความหนาของฐาน ความหนาของแขนออกมา
 - d) 4.แต่ละ joint ที่นำมาคำนวณจะไม่มีสปีลิทธิแรงเสียดทานในระบบ
 - e) 5.ความเร็วและแรงจะถูกจำลองให้มีความเร็วคงที่เสมอ

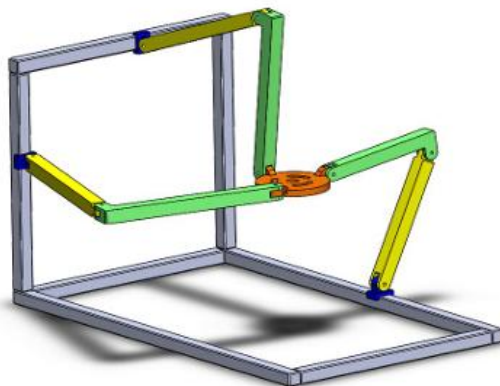
2. ทบทวนวรรณกรรม และ ทฤษฎีที่เกี่ยวข้อง (Literature Review)

[1]POSITION AND FORCE ANALYSIS OF A 3-DOF PARALLEL TRANSLATIONAL MECHANISM

Tripteron เป็นกลไกที่แต่ละแขนแยกการทำงานออกจากกันอย่างสมบูรณ์ ซึ่งหมายความว่า แอคทูเอเตอร์เชิงเส้นแต่ละตัวบนข้อต่อแบบ PRISMATIC จะให้เพียงหนึ่งองศาอิสระทางการเคลื่อน นอกจากนี้ องศาอิสระทั้งสามทั้งหมดยังขนานกับทิศทางการเคลื่อนที่ของแอคทูเอเตอร์ที่สอดคล้องกัน แต่ต่างจากกับแอคทูเอเตอร์อื่น ๆ ดังนั้น ทุ่นยนต์สามารถอธิบายได้ว่าเป็นแบบ PM การเคลื่อน 3-DOF

$$M = 6(n - 1) - \sum_{i=1}^j (6 - f_i) = 6(11 - 1) - \sum_{i=1}^{12} (6 - 1) = 0$$

โดย M คือค่าการเคลื่อนที่รวม (MOBILITY/DOF) และ n คือ จำนวนชิ้นส่วนหรือบอดี้ทั้งหมด

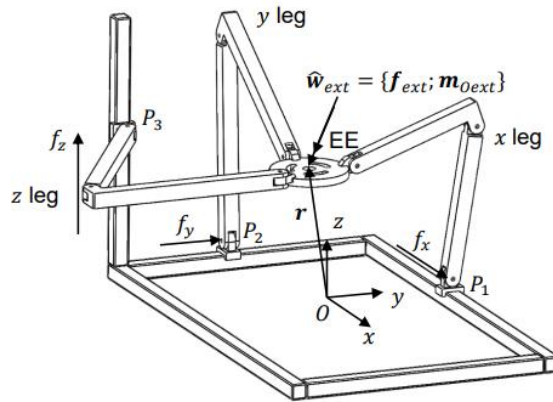


รูปที่ 1 แบบจำลอง 3 มิติของดีไซน์ TRIPTERON

พื้นที่ทำงานถูกจำกัดเป็นทรงสี่เหลี่ยมมุมฉากขนาด $l_1 \times l_2 \times l_3$ CM เมื่อต้องคำนึงถึงขนาดแพลตฟอร์มและคิเนแมติกส์ของกลไก จำเป็นต้องเลือกค่าพารามิเตอร์เฉพาะ ได้แก่ โครง (FRAME), ข้อต่อ/ลิงก์ของขา 3 ขาแต่ละท่อน และเอ็นดีเอฟเพกเตอร์

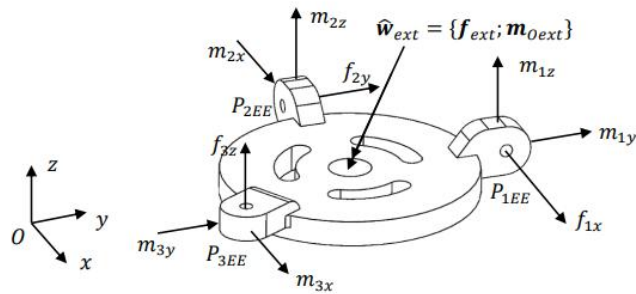
REVERSE FORCE ANALYSIS

ในการออกแบบ TRIPTERON มี 3 ACTUATOR สามารถออกแรงได้ เมื่อแรงภายนอกถูกนำมาใช้กับปลายเอฟเพกเตอร์สามารถหาค่าแรงจากตัวกระตุ้นเชิงเส้นทั้งสามได้ นอกจากนี้ยังมีแรงและโมเมนต์ตอบสนองที่ข้อต่อหมุนเกิดขึ้นเนื่องจากแรงภายนอก



รูปที่ 2 แบบจำลอง TRIPTERON เมื่อมีแรง W มากระทำต่อ EE

จากรูปมี WRENCH ที่จุด EE และได้กำหนดเวกเตอร์ตำแหน่ง R ของ EE ไว้แล้ว เพื่อให้การวิเคราะห์ง่ายขึ้น จึงไม่คิดแรงโน้มถ่วงที่กระทำกับลิงก์



รูปที่ 3 จุด EE แรงที่กระทำในแต่ละข้อ

เพื่อหาองค์ประกอบที่ยังไม่ทราบทั้งหมด ได้แสดง FBD ของ EE ไว้ในรูปที่ 4-5 เนื่องจากพารามิเตอร์ของกลไกเป็นค่าคงที่และเวกเตอร์ตำแหน่ง R ถูกกำหนดแล้ว เราจึงคำนวณพิกัดของจุด PE จาก FBD: ขาแกน X มีแรงที่พาดผ่านจุด P1EE ในทิศแกน X และ ไม่มี องค์ประกอบแรงในทิศ Y หรือ Z เพราะข้อต่อหมุนทั้งสามของขาแกน X ไม่สามารถให้ ‘แรงบิด (TORQUE)’ ที่จำเป็นในการถ่วงสมดุลแรงตาม Y และ Z ได้

$$\begin{Bmatrix} f_1 \\ m_{01} \end{Bmatrix} = \begin{Bmatrix} f_{1x} \\ 0 \\ 0 \\ 0 \\ m_{1y} + f_{1x}z_{1EE} \\ m_{1z} - f_{1x}y_{1EE} \end{Bmatrix} \quad (1)$$

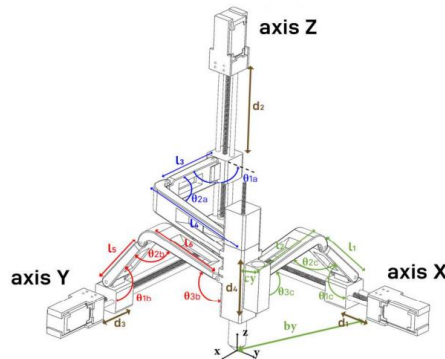
$$\begin{Bmatrix} f_2 \\ m_{02} \end{Bmatrix} = \begin{Bmatrix} 0 \\ f_{2y} \\ 0 \\ m_{2x} + f_{2xy}Z_{2EE} \\ 0 \\ m_{2z} - f_{2y}X_{2EE} \end{Bmatrix} \quad (2)$$

$$\begin{Bmatrix} f_3 \\ m_{03} \end{Bmatrix} = \begin{Bmatrix} 0 \\ 0 \\ f_{3z} \\ m_{3x} + f_{3z}Y_{3EE} \\ m_{3y} - f_{3z}X_{3EE} \\ 0 \end{Bmatrix} \quad (3)$$

การทำให้แรงบิดขาของขาเท่ากับแรงบิดภายนอกให้ผลลัพธ์ดังนี้

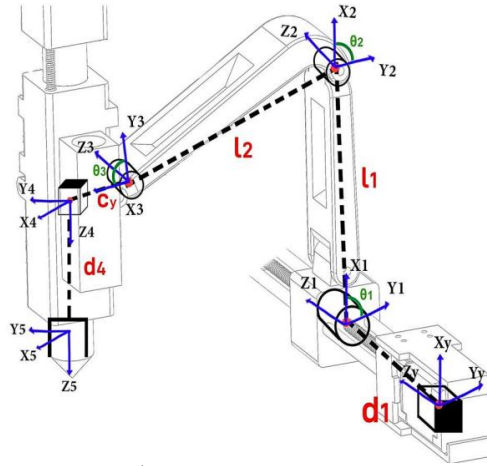
$$\begin{Bmatrix} f_{ext_x} \\ f_{ext_y} \\ f_{ext_z} \\ m_{0ext_x} \\ m_{0ext_y} \\ m_{0ext_z} \end{Bmatrix} = \begin{Bmatrix} f_{1x} \\ f_{2y} \\ f_{3z} \\ m_{2x} + m_{3x} + f_{3z}Y_{3EE} - f_{2y}Z_{2EE} \\ m_{1y} + m_{3y} + f_{1x}Z_{1EE} - f_{3z}X_{3EE} \\ m_{1z} + m_{2z} + f_{2y}X_{2EE} - f_{1x}Y_{1EE} \end{Bmatrix} \quad (4)$$

[2] MECHATRONICS DESIGN AND KINEMATIC SIMULATION OF A TRIPTERON CARTESIAN-PARALLEL AGRICULTURAL ROBOT MOUNTED ON 4-WHEELED MOBILE PLATFORM TO PERFORM SEED SOWING ACTIVITY
FORWARD KINEMATIC



รูปที่ 4 แบบจำลอง TRIPTERON

ในการคำนวณ FOWAED KINEMATIC TRIPTERON จะต้องแยกกลไกออกเป็น 3 ส่วน ในแต่ละแขนใช้ วิธี DENAVIT-HARTENBERG (DH) ในการตั้งพารามิเตอร์



รูปที่ 5 ตัวอย่างแขนในTRIPTERON

JOINT	KINEMMATIC PARMETER			
	θ_i	d_i	a_i	α_i
1	0	d_1	0	0
2	θ_2	0	l_1	0
3	θ_3	0	l_2	0
4	θ_4	0	c_Y	90°
5	0	d_2	0	0

ตารางที่ 1 DENAVIT-HARTENBERG (DH) PARAMETER ของแขน

d_i คือระยะการเลื่อนที่แปรผันของข้อต่อที่ i , θ_i คือมุมการหมุนที่แปรผันของข้อต่อที่ i , l_1 คือความยาวลิงก์ระหว่างข้อต่อ 2 และ 3
 l_2 คือความยาวลิงก์ระหว่างข้อต่อ 3 และ 4, c_Y คือระยะจากข้อต่อ 4 ไปยังจุดกึ่งกลางของข้อต่อ 5

$$T_{\text{partial} - Y} = A_Y^5 = A_Y^1 A_1^2 A_2^3 A_3^4 A_4^5 \quad (5)$$

โดยที่ A คือเมทริกซ์ทรานส์ฟอร์มโฮโมจีเนียสที่เชื่อมระบบพิกัด I-1 กับระบบ I และ A_1Y คือเมทริกซ์ที่เชื่อมระบบฐานของโซ่แขน Y กับระบบที่ 1

$$T_y = T_{\text{base} \rightarrow y} A_Y^1 A_1^2 A_2^3 A_3^4 A_4^5 \quad (6)$$

$T_{\text{base} \rightarrow Y}$ คือเมทริกซ์ทรานส์ฟอร์มแบบโฮโมจีเนียสที่เชื่อมระบบพิกัด XYZ กับระบบพิกัดบนแกน Y

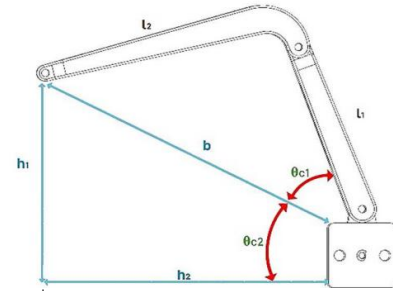
$$T_x = T_{\text{base} - x} A_x^6 A_6^7 A_7^8 A_8^9 A_9^5 \quad (7)$$

$$T_z = T_{\text{base} - z} A_z^{10} A_{10}^{11} A_{11}^{12} A_{12}^{13} A_{13}^5 \quad (8)$$

$T_{base \rightarrow X}$ หรือ Z คือเมทริกซ์ที่เปลี่ยนจากฐานกลาง XYZ ไปยังฐานของโซ่แกน X หรือ Z

INVERSE KINEMATIC

ในการจะหา JOINT ที่เปลี่ยนไปจากการเลื่อน LINK จะต้องหาจาก INVERSE KINEMATIC โดยจากรายงานที่ได้มาเราสามารถคำนวณมุม $\theta_{c1} \theta_{c2}$ ของมุมต่างๆได้จากสมการดังนี้ ตัวอย่างนี้นำเสนอการคำนวณผ่านแกน X



รูปที่ 6 Free body diagram ของ TRIPTERON

$$\theta_{c1} = \cos^{-1}\left(\frac{b^2 + l_1^2 - l_2^2}{2 * b * l_1}\right) \quad (9)$$

$$\theta_{c2} = \tan^{-1}\left(\frac{h_1}{h_2}\right) \quad (10)$$

โดยองศาทั้งหมดที่ได้จะเท่ากับ

$$\theta_{total} = \cos^{-1}\left(\frac{b^2 + l_1^2 - l_2^2}{2 * b * l_1}\right) + \tan^{-1}\left(\frac{h_1}{h_2}\right) \quad (11)$$

จากที่เห็นจากรูปภาพที่ 4 และ รูปภาพที่ 6 จะทำให้เห็นจุด EE ห่างกับจุด ACTUATOR เท่ากับ b และสามารถเคลื่อนที่ได้ d จาก ACTUATOR โดย $l_1 l_2$ คือ LINK บน และ LINK ล่าง ที่ทำการเชื่อมระหว่าง EE กับ ACTUATOR ซึ่งสามารถใช้ในการหา ตำแหน่ง ความเร็ว และ ความเร่งได้จากสมการนี้

$$b_Y + d_1 + c_Y + l_1 + l_2 = [X \ Y \ Z]^T \quad (12)$$

เนื่องจาก b อยู่ในแกนแนว Y และ $l_1 l_2$ จึงตั้งฉากกับแกน X เมื่อนำมา Dot product เมื่อ $\hat{i}$ ตำแหน่งของ X-LINEAR JOINT สามารถแสดงได้จากสมการนี้

$$d_1 = (X - b_x \hat{i}) \hat{i} \quad (13)$$

นำสมการที่ 12 มาใส่สมการที่ 13

$$\lambda_x = l_1 + l_2 \quad (14)$$

$$\lambda_x = [0 \ Y \ Z]^T - b_x - c_x + (b_x \hat{i}) \quad (15)$$

จากสมการที่ 11 เราสามารถแทนค่าลงสมการที่เพื่อที่ได้สมการดังนี้

$$a_x = \cos^{-1} \left(\frac{||\lambda_x||^2 + ||l_1||^2 - ||l_2||^2}{2 * ||l_1||^2 * ||\lambda_x||^2} \right) + \tan^{-1} \left(\frac{z - b_x * \hat{k}}{Y - (b_x + c_x) * \hat{j}} \right) \quad (16)$$

ดังนั้น เมื่อคำนวณ a_x แล้ว จะได้ l_1 ดังนี้:

$$l_1 = l_1(\cos a_x \hat{j} + \sin a_x \hat{k}) \quad (17)$$

และเวกเตอร์ L_1 สามารถหาได้โดยการแทรกสมการที่ 17 ลงในสมการที่ 14

$$l_2 = \lambda_x - l_1(\cos a_x \hat{j} + \sin a_x \hat{k}) \quad (18)$$

เมื่อแก้ปัญหานี้ทำใน LINK ที่ 1 ซึ่งตั้งอยู่บนแกน X และทำซ้ำใน LINK ที่ 2 (แกน Y) และ LINK ที่ 3 (แกน Z) ของหุ่นยนต์ TRIPTERON ด้วยวิธีการนี้ จะได้ค่าดังนี้

$$d_3 = (Y - b_Y \hat{j}) \hat{j} \quad (19)$$

$$l_5 = l_5(\cos a_Y \hat{j} + \sin a_Y \hat{k}) \quad (20)$$

$$a_x = \cos^{-1} \left(\frac{||\lambda_Y||^2 + ||l_5||^2 - ||l_6||^2}{2 * ||l_5||^2 * ||\lambda_Y||^2} \right) + \tan^{-1} \left(\frac{z - b_Y * \hat{k}}{(b_Y) * \hat{i} - X} \right) \quad (21)$$

$$\lambda_x = [X \ 0 \ Z]^T - b_Y + (b_Y \hat{j}) \quad (22)$$

$$l_5 = \lambda_Y - l_6(\cos a_Y \hat{j} + \sin a_Y \hat{k}) \quad (23)$$

$$d_2 = (z - b_Z \hat{k}) \hat{k} \quad (24)$$

$$l_3 = l_3(\cos a_Z \hat{j} - \sin a_Z \hat{k}) \quad (25)$$

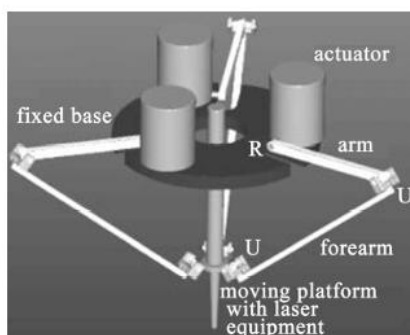
$$a_x = \cos^{-1} \left(\frac{||\lambda_Z||^2 + ||l_3||^2 - ||l_4||^2}{2 * ||l_3||^2 * ||\lambda_Z||^2} \right) + \tan^{-1} \left(\frac{b_Z * \hat{i} - X}{(b_Z * \hat{j} - Y)} \right) \quad (26)$$

$$\lambda_Z = [X \ Y \ 0]^T - b_Z + (b_Z \hat{k}) \quad (27)$$

$$l_4 = \lambda_Y - l_3(\cos a_Z \hat{j} + \sin a_Z \hat{k}) \quad (28)$$

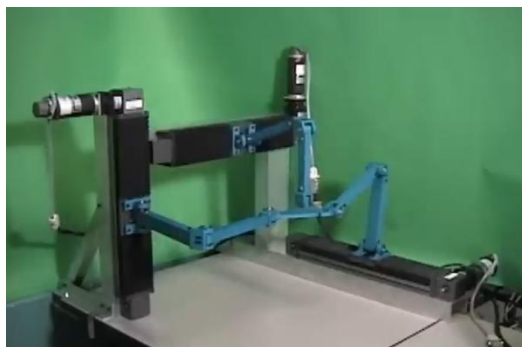
[3].CLOSE-LOOP MANIPULATOR

โดยทั่วไปจะหมายถึง PARALLEL MANIPULATOR (หุ่นยนต์แขนกลแบบขนาน) เป็นกลไกที่มีการเชื่อมโยงระหว่างส่วนฐาน (BASE) และปลายมือ (END-EFFECTOR) ผ่านแขนกลจลนศาสตร์หลายชุด (MULTIPLE KINEMATIC CHAINS) ซึ่งแต่ละแขนทำงานร่วมกันในลักษณะ วงปิด (CLOSED KINEMATIC CHAIN) และขับเคลื่อนเพื่อรักษาสมดุลของโครงสร้างทั้งหมดให้คงรูปอยู่เสมอ



รูปที่ 7 ตัวอย่างหุ่นยนต์ PARALLEL CLOSED-LOOP MANIPULATOR มีโครงสร้างลักษณะเป็นวงปิด ซึ่งมี 3 แขนเชื่อมฐานและปลายมือ

สำหรับหุ่นยนต์ TRIPTERON ที่เป็นหุ่นยนต์แบบขนาน (PARALLEL MANIPULATOR) มี โครงสร้างจลนศาสตร์แบบวงปิด (CLOSED-LOOP KINEMATIC CHAIN MECHANISM) หรือก็คือเมื่อผู้ควบคุมสั่งให้ปลายมือของ TRIPTERON เคลื่อนที่ในแนวแกนใดแกนหนึ่ง เช่น แกน X ตัวขับเคลื่อนในแกนแกน X จะเคลื่อนออก ขณะที่แกน Y และ Z จะปรับระยะเชื่อมโยงเพื่อคงรูปร่างของโครงสร้างไว้ ผลลัพธ์คือ ปลายมือเคลื่อนที่เป็นเส้นตรงในแนว X โดยไม่เกิดการเอียงหรือหมุน ซึ่งเป็นลักษณะเฉพาะของระบบแบบวงปิดใน TRIPTERON ซึ่งหมายความว่า การเคลื่อนที่ของปลายมือ (END-EFFECTOR) ถูกควบคุมและรองรับโดยโครงสร้างแขนกลหลายชุดที่เชื่อมต่อกันเป็นวงปิดระหว่างฐาน (BASE) ในส่วนโครงสร้างของ TRIPTERON แขนกลทั้งสามชุดถูกจัดวางในแนวตั้งฉากกันตามแกน X, Y และ Z แต่ละแขนจะให้การเคลื่อนที่ในแนวเชิงเส้นหนึ่งทิศทาง (TRANSLATIONAL DOF) โดยรวมแล้วหุ่นยนต์นี้สามารถเคลื่อนที่ได้สามองศาอิสระเชิงเส้น (3-DOF TRANSLATIONAL MOTION)



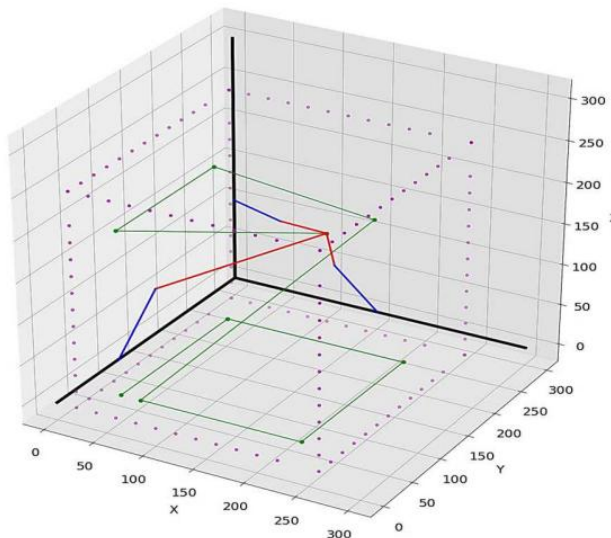
รูปที่ 8 ที่สภาวะสมดุลของ TRIPTERON



รูปที่ 9 การเคลื่อนที่แนวแกน X ส่งผลให้ข้อต่อในแนวแกน Y , Z ขยับเพื่อรักษาสมดุลของโครงสร้าง

[4].MATLAB SIMULATION

การจำลอง (SIMULATION) เป็นกระบวนการทางคณิตศาสตร์และวิศวกรรมที่ใช้ในการศึกษาพฤติกรรมของระบบจริงโดยไม่ต้องสร้างต้นแบบจริงขึ้นมา ซึ่งช่วยลดเวลาและต้นทุนในการออกแบบระบบอย่างมาก สำหรับ (PARALLEL ROBOT) อย่าง TRIPTERON ROBOT ซึ่งมีลักษณะเฉพาะคือ การเคลื่อนที่ของปลายมือ (END-EFFECTOR) เกิดจากการทำงานร่วมกันของขาเชิงเส้นสามขาในแนวแกน X, Y และ Z



รูปที่ 10 SIMULATE MATLAB

การสร้างแบบจำลองจลนศาสตร์ (KINEMATIC MODELING)

ใช้สำหรับคำนวณหาตำแหน่งและทิศทางของปลายมือจากค่าตำแหน่งของข้อต่อที่ทราบจาก IK โดยอาศัยสมการแปลงพิกัด (TRANSFORMATION MATRIX) และหลักการ D-H PARAMETERS

การสร้างแบบจำลองจลนศาสตร์เชิงกลับ (INVERSE KINEMATICS)

ในการจำลองหรือคำนวณจลนศาสตร์เชิงกลับของ TRIPTERON เมื่อกำหนดให้ปลายมือเคลื่อนที่ในทิศทางใดทิศทางหนึ่ง จะต้องพิจารณาผลกระทบต่องานในระบบเพราะ จะทำให้ข้อต่อของอีกสองขาเกิดการเปลี่ยนตำแหน่งโดยอัตโนมัติ เพื่อรักษาสถิตและทิศทางของโครงสร้างทั้งหมด

การวิเคราะห์จาโคเบียน (JACOBIAN ANALYSIS):

MATLAB สามารถใช้ในการคำนวณเมทริกซ์จาโคเบียนของระบบ เพื่อหาความสัมพันธ์ระหว่างความเร็วของข้อต่อกับความเร็วของปลายมือ

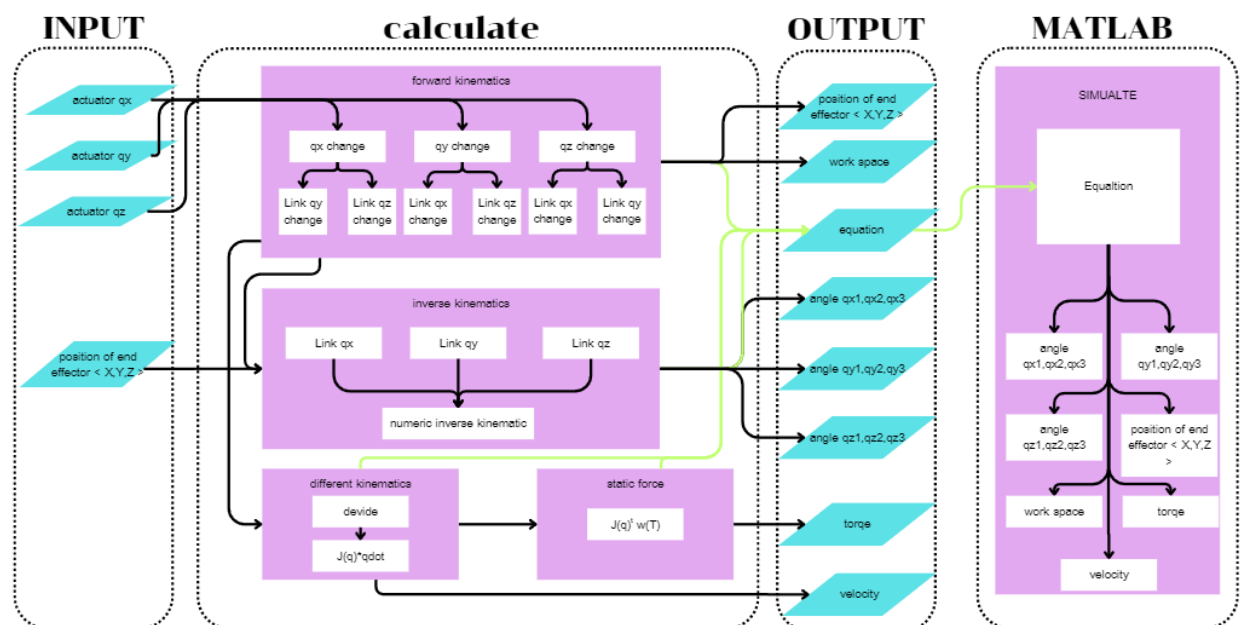
การคำนวณแรงและทอร์ก (STATIC FORCE SIMULATION)

สำหรับ TRIPTERON ROBOT ซึ่งเป็นหุ่นยนต์ขนานสามองศาอิสระแต่ละแขนจะทำงานร่วมกันเพื่อให้ EE เคลื่อนที่แบบเชิงเส้นในแนวแกน X,Y,Z โดยไม่เกิดการหมุนเมื่อมีการขับเคลื่อนขาใดขาหนึ่งอีก2ขาจะขยับเพื่อรักษาสถิตซึ่งจะก่อให้เกิดแรงปฏิกิริยา (REACTION FORCE) ระหว่างลิงก์และข้อต่อของแขนทั้งสาม ซึ่งเป็แรง STATIC FORCE ที่กระทำต่อแต่ละข้อต่อ

3. เนื้อหาในรายวิชาที่เกี่ยวข้อง

- 1) Forward kinematic
- 2) Inversed kinematic
- 3) Different kinematic
- 4) Static force

4. System Diagram / System Overview (Function and Argument)



รูปที่ 7 System Diagram

1. การคำนวณ Forward kinematic จากการขยับ actuator X Y Z ส่งผลต่อ end effector และ Joint ใน link อื่นอย่างไร
 คณะผู้จัดทำจะทำการคำนวณ Forward kinematic โดยใช้ DH-Parameter ในการดูผลกระทบต่อ Link ที่ actuator ขยับ จากนั้นจะหาตำแหน่ง end-effector ณ ปัจจุบันที่เปลี่ยน นำมาคำนวณหาระยะห่าง Link อื่น กับ end-effector ใหม่ที่เปลี่ยนไปจากเดิม แล้วใช้ inverse kinematic ในการหา q_1, q_2, q_3 ของ link นั้นเพื่อดูการเปลี่ยนแปลงของทั้งระบบจากการเปลี่ยน actuator
2. การคำนวณ Inverse Kinematic ใช้หา q_1, q_2, q_3 ของทุก Link จาก end effector ที่ได้รับเข้ามาเพื่อดูว่า Joint ใน link เปลี่ยนแปลงอย่างไร
 คณะผู้จัดทำจะนำ end-effector ที่เปลี่ยนแปลงไปนำมาหา q_1, q_2, q_3 ของ link ต่างโดยทำให้ความยาวของระยะห่างของ Link และ end-effector มีขนาดสมเหตุผลเพื่อใช้ในการดูการเปลี่ยนแปลงของทั้งระบบจากการเปลี่ยน actuator
3. การคำนวณหา different kinematic เพื่อหาความเร็วปลายมือสัมพัทธ์
 คณะผู้จัดทำจะเอา position ที่ได้จากการทำ Forward Kinematic นำมา divide ด้วยสมการแล้วทำให้ออกมาในรูปแบบของ Jacobian และ dot product กับ $\dot{q}$ จะได้ความเร็ว Jacobian ที่ได้จะนำไปใช้ในการคำนวณอย่างอื่นต่อ
4. การคำนวณหา static force เพื่อหาความเร็วปลายมือสัมพัทธ์
 คณะผู้จัดทำจะเอา คณะผู้จัดทำจะเอา Jacobian มาทำ transpose และ dot product กับ แรงที่เกิดขึ้นภายใน Link เพื่อหา torque ที่เกิดขึ้นในแต่ละ joint
5. การทำ Simulate ใน MATLAB
 หลังจากการคำนวณหา Forward kinematic , inverse kinematic , different kinematic และ static force จะได้สมการออกมาทางคณะผู้จัดทำจะทำการนำสมการเหล่านี้มาสร้าง Simulate ใน MATLAB

5. ผลการศึกษาที่คาดหวัง

- 1) เพื่อศึกษาการเคลื่อนที่ของมุมในส่วนของ Link X, Link Y และ Link Z ที่สัมพันธ์กับการเคลื่อนที่ของ End Effector
- 2) เพื่อคำนวณหามุมของลิงก์ทั้งสามที่เปลี่ยนแปลงไปจากการเคลื่อนไหวของลิงก์อื่น ๆ ภายในระบบ
- 3) เพื่อหาความเร็วสัมพัทธ์ของปลายมือ (End Effector Velocity) และสร้างเมทริกซ์ Jacobian ของระบบเพื่อใช้ในการคำนวณทางจลนศาสตร์และการควบคุม
- 4) เพื่อคำนวณแรงสถิต (Static Force) ที่กระทำต่อข้อต่อ (Joint) ทั้งหมดในระบบ
- 5) เพื่อศึกษาการทำการ simulate ภายในระบบ MATLAB

6. กระบวนการคำนวณ Tripteron

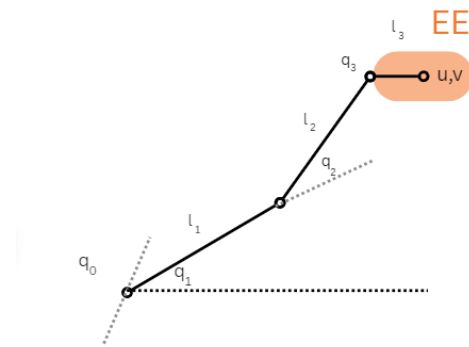
1. วิเคราะห์โครงสร้างเชิงกล (Mechanics) ของระบบ Tripteron

ระบบ Tripteron เป็นหุ่นยนต์แบบ Parallel Manipulator ซึ่งประกอบด้วยขาทั้งหมดสามขา ได้แก่ X-leg, Y-leg และ Z-leg โดยแต่ละขามีโครงสร้างภายในแบบเดียวกันทั้งหมด คือประกอบด้วยข้อต่อแบบ Prismatic จำนวน 1 ตัว และข้อต่อแบบ Revolute จำนวน 3 ตัว Planar 3R + 1 Prismatic หรือเรียกว่าโครงสร้าง P-R-R-R เมื่อรวมทั้งสามขาแล้ว หุ่นยนต์ Tripteron จะมีจำนวนข้อต่อทั้งหมด 9 ตัว (3P + 6R) ปลายลิงก์สุดท้ายของแต่ละขามาบรรจบกันที่ End-Effector (EE) ทำงานร่วมกันแบบขนานเพื่อควบคุมตำแหน่ง โดยมีหลักการทำงานดังนี้

- 1.1 **Prismatic Joint(P)** ทำหน้าที่เป็นตัวเลื่อนในทิศแกนหลักของขาโดยขา X,Y,Z จะเลื่อนในแกนนั้นๆ เช่น ขา X จะมี Prismatic Joint เคลื่อนไปตามแกน X ส่วนขา Y และขา Z ก็จะมีการเคลื่อนที่ตามแกนของตนเองตามลำดับซึ่งการเคลื่อนของ P จะกำหนด ตำแหน่งฐาน ของ planar 3R ของแต่ละขา ทำให้สามารถกำหนดตำแหน่ง EE ใน 3D space ได้

- 1.2 **Revolute Joints(R)** ขาของ Tripteron จะมีข้อต่อแบบหมุน 3 ตัวโดย R1 คือ ข้อต่อข้อขาที่เชื่อมต่อ P และ R1 ส่วนของ R2 คือข้อต่อของขาที่ต่อ L1 กับ L2 ส่วนของ R3 ที่ต่อ L2 และ End-Effector(EF) ซึ่งทิศการหมุนของข้อต่อของแต่ละขาจะเป็นหมุนทิศเดียวกันกับแกนนั้นๆ
- 1.3 **End-Effector(EF)** EF ของ Tripteron อยู่ตำแหน่งศูนย์กลางที่ปลายขา X, Y และ Z มาบรรจบกัน ในแบบจำลองนี้ มีการกำหนดค่า offset จากลิงก์ L2 สู่ EF เป็นความยาว L3
- 1.4 **ระนาบการเคลื่อนที่ของแต่ละขา** workspace ของแต่ละขามีรูปร่างกลม (planar), และเมื่อรวมสามขา เกิดเป็น cubic workspace แต่ละขาทำงานภายในระนาบ (Plane) ที่ไม่ทับซ้อนกัน เพื่อให้เกิดการเคลื่อนที่เป็นอิสระ และทำให้ทั้งสามขาร่วมกันควบคุมตำแหน่งของ End-Effector ได้โดยไม่ชนกัน โครงสร้างของแต่ละขาจึงถูกจัดให้อยู่ในระนาบดังนี้ ขา X ถูกออกแบบให้ขยับในระนาบ Y-Z ขา Y ถูกออกแบบให้ขยับในระนาบ X-Z ขา Z ถูกออกแบบให้ขยับในระนาบ X-Y

2.forward kinematic ของแต่ละแขน



รูปที่ 8 free body ของขาในแต่ละแกน

ทำการคำนวณ forward kinematics $FK(q) = U, V$ จะได้

$$u = L_1 C_1 + L_2 C_{12} + L_3 C_{123}$$

$$v = L_1 S_1 + L_2 S_{12} + L_3 S_{123}$$

ซึ่งแต่ละแขนจะมี Transformation matrix ดังนั้นแกน X จะใช้ DH Parameter มาใช้ในการคำนวณมาประยุกต์ใช้ดังนี้

แกน x จะทำการเปลี่ยนจากเลื่อนตำแหน่งแกน X ไป แกน Z และ หมุนในแกน Z แทนส่วนการเลื่อนตำแหน่งในแกน Z จะเปลี่ยนไปเลื่อนในแนวแกน X และ หมุนในแนวแกน Z และหมุนตามเข็มนาฬิกา ซึ่งจะตอบ matrix ดังนี้

N-1	N	a	α	d	θ
0	1	0	0	dx	q_{1x}
1	2	L_{1x}	0	0	q_{2x}
2	3	L_{2x}	0	0	q_{3x}
3	e	L_{3x}	0	0	0

ตารางที่ 2 DH ของแขน X

$$T_x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & a \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} c_\alpha & -s_\alpha & 0 & 0 \\ s_\alpha & c_\alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & d \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c_\theta & s_\theta & 0 \\ 0 & -s_\theta & c_\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (28)$$

$$T_x = \begin{bmatrix} c_\alpha & -s_\alpha c_\theta & -s_\alpha s_\theta & dc_\alpha \\ s_\alpha & c_\alpha c_\theta & c_\alpha s_\theta & ds_\alpha \\ 0 & -s_\theta & c_\theta & a \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (29)$$

(30)

$$T_x^0 = T_x^0 T_x^1 T_x^2 T_x^3$$

$$T_x^0 = \begin{bmatrix} 1 & 0 & 0 & dx \\ 0 & c_{q1x+q2x+q3x} & s_{q1x+q2x+q3x} & L_{1x}s_{q1x} + L_{2x}s_{q1x+q2x} + L_{3x}s_{q1x+q2x+q3x} \\ 0 & -s_{q1x+q2x+q3x} & c_{q1x+q2x+q3x} & L_{1x}c_{q1x} + L_{2x}c_{q1x+q2x} + L_{3x}c_{q1x+q2x+q3x} \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (31)$$

แกน y การเลื่อนตำแหน่งในแกน Z จะเปลี่ยนไปเลื่อนในแนวแกน y และ หมุนในแนวแกน y และหมุนตามเข็มนาฬิกา ซึ่งจะ
ตอบ matrix ดังนี้

N-1	N	a	α	d	θ
0	1	0	0	dy	0
1	2	L_{1y}	0	0	q_{1y}
2	3	L_{2y}	0	0	q_{2y}
3	e	L_{3y}	0	0	q_{3y}

ตารางที่ 3 DH ของแขน Y

$$T_y = \begin{bmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c_\alpha & -s_\alpha & 0 \\ 0 & s_\alpha & c_\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & d \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} c_\theta & 0 & -s_\theta & 0 \\ 0 & 1 & 0 & 0 \\ s_\theta & 0 & c_\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (32)$$

$$T_y = \begin{bmatrix} c_\theta & 0 & -s_\theta & a \\ -s_\alpha s_\theta & c_\alpha & -s_\alpha c_\theta & dc_\alpha \\ c_\alpha s_\theta & s_\alpha & c_\alpha c_\theta & ds_\alpha \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (33)$$

$$T_y^0 = T_y^0 T_y^1 T_y^2 T_y^3 \quad (34)$$

$$T_y^0 = \begin{bmatrix} c_{q1y+q2y+q3y} & 0 & -s_{q1y+q2y+q3y} & L_{1y}c_{q1y} + L_{2y}c_{q1y+q2y} + L_{3y}c_{q1y+q2y+q3y} \\ 0 & 1 & 0 & dy \\ s_{q1y+q2y+q3y} & 0 & c_{q1y+q2y+q3y} & -L_{1y}s_{q1y} - L_{2y}s_{q1y+q2y} - L_{3y}s_{q1y+q2y+q3y} \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (35)$$

แกน z จะใช้จะเปลี่ยนแค่ rotation ใน แกน z ให้หมุนตามเข็ม

N-1	N	a	α	d	θ
0	1	0	0	dz	0
1	2	L_{1z}	0	0	q_{1z}
2	3	L_{2z}	0	0	q_{2z}
3	e	L_{3z}	0	0	q_{3z}

ตารางที่ 4 DH ของแขน Z

$$T_z = \begin{bmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c_\alpha & -s_\alpha & 0 \\ 0 & s_\alpha & c_\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} c_\theta & s_\theta & 0 & 0 \\ -s_\theta & c_\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (36)$$

$$T_z = \begin{bmatrix} c_\theta & s_\theta & 0 & a \\ -c_\alpha s_\theta & c_\alpha c_\theta & -s_\alpha & -ds_\alpha \\ -s_\alpha s_\theta & s_\alpha c_\theta & c_\alpha & dc_\alpha \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (37)$$

$$T_z^0 = T_z^1 T_z^2 T_z^3 T_z^e \quad (38)$$

$$T_y^0 = \begin{bmatrix} c_{q_{1z}+q_{2z}+q_{3z}} & s_{q_{1z}+q_{2z}+q_{3z}} & 0 & L_{1z}c_{q_{1z}} + L_{2z}s_{q_{1z}+q_{2z}} + L_{3z}s_{q_{1z}+q_{2z}+q_{3z}} \\ -s_{q_{1z}+q_{2z}+q_{3z}} & c_{q_{1z}+q_{2z}+q_{3z}} & 0 & -L_{1z}s_{q_{1z}} - L_{2z}s_{q_{1z}+q_{2z}} - L_{3z}s_{q_{1z}+q_{2z}+q_{3z}} \\ 0 & 0 & 1 & dz \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (39)$$

3.inverse kinematic ของแต่ละ joint

ในการหา inverse kinematic เพื่อใช้ในการคำนวณหาองศาของแต่ละ joint โดยการหาจะต้องกำหนดได้จาก ตำแหน่งสุดท้ายที่ end-effector อยู่และองศาของของแต่ละแกน โดยแต่ละแกนจะถูกกำหนดได้เนื่องจาก end-effector ไม่สามารถเกิดการหมุนได้เนื่องจาก moment ภายใน end-effector เป็น 0 โดยที่องศาของแกน x คือ 0 ของแกน y คือ 0 และของแกน z คือ 180 องศาโดยวิธีการหา inverse kinematic จะหาตามสมการนี้

3.1.หาองศาที่ q_2

θ คือ องศา

X, Y คือ ตำแหน่งสุดท้ายของ link2

P_x, P_y คือ ตำแหน่งสุดท้ายของ end-effector

$$X = P_x - L_3 \cos \theta \quad (40)$$

$$Y = P_y - L_3 \sin \theta \quad (41)$$

$$\cos q_2 = \frac{X^2 + Y^2 - L_1^2 - L_2^2}{2 \times L_1 \times L_2} \quad (42)$$

$$\sin q_2 = \sqrt{1 - (\cos q_2)^2} \quad (43)$$

$$q_2 = \text{atan2}(\pm \sin q_2, \cos q_2) \quad (44)$$

3.2. หองศาที่ q_1

$$K_1 = L_1 + L_2 \cos q_1 \quad (45)$$

$$K_2 = L_2 \sin q_1 \quad (46)$$

$$r = \sqrt{k_1^2 + k_2^2} \quad (47)$$

$$\cos(q_1 + \gamma) = \frac{X}{r} \quad (48)$$

$$\sin(q_1 + \gamma) = \frac{Y}{r} \quad (49)$$

$$\gamma = \text{atan2}(k_2, k_1) \quad (50)$$

$$q_1 = \text{atan2}\left(\frac{Y}{r}, \frac{X}{r}\right) - \gamma \quad (51)$$

3.2. หองศาที่ q_3

$$q_3 = \theta - q_1 - q_2 \quad (52)$$

4. Jacobian และการคำนวณ static torque ของแต่ละแขน

หลังจากที่ได้ forward kinematics มาแล้วจะหา matrix ของ Jacobian ทำโดยอนุพันธ์ u และ v เทียบ q_1, q_2, q_3 ตามสมการนี้

$$J = \begin{bmatrix} \frac{\partial u}{\partial q_1} & \frac{\partial u}{\partial q_2} & \frac{\partial u}{\partial q_3} \\ \frac{\partial v}{\partial q_1} & \frac{\partial v}{\partial q_2} & \frac{\partial v}{\partial q_3} \end{bmatrix} \quad (53)$$

จะได้ matrix 2x3 ซึ่งขาของ Tripteron มี q_0 ที่เป็นแกนเลื่อนด้วยเราจึงต้องจัดระนาบของขาในแต่ละแกนให้ถูกต้อง

แปลงระนาบขาแกน x

$$\begin{aligned} X &= q_0 \\ Y &= U \\ Z &= V \end{aligned}$$

ดังนั้น matrix Jacobian ของขา X จะเท่ากับ

$$J_x = \begin{bmatrix} 0 & 0 & 0 & 1 \\ \frac{\partial u}{\partial q_1} & \frac{\partial u}{\partial q_2} & \frac{\partial u}{\partial q_3} & 0 \\ \frac{\partial v}{\partial q_1} & \frac{\partial v}{\partial q_2} & \frac{\partial v}{\partial q_3} & 0 \end{bmatrix} \quad (54)$$

$$J_x = \begin{bmatrix} 0 & 0 & 0 & 1 \\ -L_1 S_1 - L_2 S_{12} - L_3 S_{123} & -L_2 S_{12} - L_3 S_{123} & -L_3 S_{123} & 0 \\ L_1 C_1 + L_2 C_{12} + L_3 C_{123} & L_2 C_{12} + L_3 C_{123} & +L_3 C_{123} & 0 \end{bmatrix} \quad (55)$$

แปลงระนาบขาแกน y

$$\begin{aligned} X &= U \\ Y &= q_0 \\ Z &= V \end{aligned}$$

ดังนั้น matrix Jacobian ของขา Y จะเท่ากับ

$$J_y = \begin{bmatrix} \frac{\partial u}{\partial q_1} & \frac{\partial u}{\partial q_2} & \frac{\partial u}{\partial q_3} & 0 \\ 0 & 0 & 0 & 1 \\ \frac{\partial v}{\partial q_1} & \frac{\partial v}{\partial q_2} & \frac{\partial v}{\partial q_3} & 0 \end{bmatrix} \quad (56)$$

$$J_y = \begin{bmatrix} -L_1 S_1 - L_2 S_{12} - L_3 S_{123} & -L_2 S_{12} - L_3 S_{123} & -L_3 S_{123} & 0 \\ 0 & 0 & 0 & 1 \\ L_1 C_1 + L_2 C_{12} + L_3 C_{123} & L_2 C_{12} + L_3 C_{123} & +L_3 C_{123} & 0 \end{bmatrix} \quad (57)$$

แปลงระนาบขาแกน z

$$\begin{aligned} X &= U \\ Y &= V \\ Z &= q_0 \end{aligned}$$

ดังนั้น matrix Jacobian ของขา Z จะเท่ากับ

$$J_z = \begin{bmatrix} \frac{\partial u}{\partial q_1} & \frac{\partial u}{\partial q_2} & \frac{\partial u}{\partial q_3} & 0 \\ \frac{\partial v}{\partial q_1} & \frac{\partial v}{\partial q_2} & \frac{\partial v}{\partial q_3} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (58)$$

$$J_z = \begin{bmatrix} -L_1 S_1 - L_2 S_{12} - L_3 S_{123} & -L_2 S_{12} - L_3 S_{123} & -L_3 S_{123} & 0 \\ L_1 C_1 + L_2 C_{12} + L_3 C_{123} & L_2 C_{12} + L_3 C_{123} & L_3 C_{123} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (59)$$

การคำนวณ static torque จะใช้สูตร

$$\tau = J^T F \quad (60)$$

โดยที่ $F = \begin{bmatrix} Fx \\ Fy \\ Fz \end{bmatrix}$ และ torque ที่เป็น Matrix ขนาด 4x1 torque $\tau = \begin{bmatrix} \tau_0 \\ \tau_1 \\ \tau_2 \\ \tau_3 \end{bmatrix}$ ของ q0 (prismatic) q1, q2,

q3 (Revolute)

5. Simulate ใน MATLAB

5.1 ขั้นตอนการทำ simulate ผ่าน MATLAB

5.1.1. การกำหนดค่าเริ่มต้นและการหา Home Configuration

กำหนด Parameter เบื้องต้น: ผู้ใช้งานจะทำการกำหนด parameter ต่างๆ ของระบบหุ่นยนต์ เช่น ความยาวของแขน ข้อต่อ (links) และค่าอื่นๆ ที่เกี่ยวข้องตามความต้องการ

5.1.2. การคำนวณหาองศาข้อต่อสำหรับการเคลื่อนที่ทั้งหมด

ในการ simulate ระบบจะต้องมี home configuration ของ Tripteron ซึ่งสามารถหาได้จากการทำ inverse kinematic โดยใช้หาองศาตลอดการเคลื่อนที่ทั้งหมดได้จึงจะทำการเก็บค่าตั้งแต่องศาต้นไปถึงองศาสุดท้ายที่หุ่นจะเคลื่อนที่

5.1.3. การหาตำแหน่ง joint ผ่าน Forward Kinematics

คำนวณตำแหน่ง Joint โดยการนำองศาข้อต่อที่จัดเก็บไว้ในขั้นตอนที่ 2 มาคำนวณผ่าน Forward Kinematics (FK) เพื่อหา ตำแหน่งของ joint แต่ละแกนแขนที่แท้จริงใน work space

5.1.4. การจำลองการเคลื่อนที่ของ Tripteron

เมื่อได้ตำแหน่ง position ของทุก joint แล้วจะนำมา plot 3D ผ่าน MATLAB ซึ่งจะจำลองการเคลื่อนที่ทั้งหมดของตัวหุ่นตลอดระยะเวลาที่ได้ตั้งเอาไว้

5.2 กำหนด parameter

เพื่อให้กลไก Planar 3R ของแต่ละขาบรรจบกันได้อย่างเหมาะสม ความยาวลิงก์ และ workspace จะถูกจัดให้สมมาตรทุกขา ดังนี้ Work space ที่ prismatic joint สามารถเคลื่อนที่ได้จะกำหนดให้

5.1.1 แกน X มีค่า 0.5 – 4.5 m. Y มีค่า 0.5 -4 m. Z มีค่า 0.5 – 4 m.

5.1.2 ความยาวของ Link โดย L1 ยาว 2.8284 m. L2 ยาว 2.8284 m. และ offset EE L3 ยาว 0.5m.

5.1.3 ตำแหน่งเริ่มต้นของ EE สามารถกำหนดได้เอง dx ,dy ,dz แต่จะต้องอยู่ใน work space ที่ตั้งเอาไว้

5.1.4 ตำแหน่งสุดท้ายของ EE สามารถกำหนดได้เอง New_dx ,New_dy ,New_dz แต่จะต้องอยู่ใน work space ที่ตั้งเอาไว้

5.1.5 การกำหนดแรงที่ EE สำหรับรายงานนี้เราได้กำหนดทิศทางแรงที่กดเข้าไปดังนี้ Fx Fy และ Fz ซึ่งสามารถกำหนดได้เอง

สำหรับการกำหนดตำแหน่งจาก mechanic ของ Tripteron แขนตำแหน่งแกน prismatic จะอิงจาก EE เสมอการกำหนดตำแหน่ง EE = ระยะเลื่อนของแกน prismatic

ค่าพารามิเตอร์ทั้งหมดจะถูกใช้เป็นข้อมูลตั้งต้นในการคำนวณ Inverse Kinematics, Forward Kinematics, Jacobian, และ Static Force

```
% Robot link lengths
L1 = 2*(sqrt(2));
L2 = 2*(sqrt(2));
L3 = 0.5;
dx = 2.5;
dy = 2.5;
dz = 2.5;
New_dx = 4.5;
New_dy = 0;
New_dz = 4;
simulate = 1;
ro = 0;
%recomendation workspace
% 0.5 < dx < 4.5
% 0.5 < dy < 4
% 0.5 < dz < 4
% ส่วนstatic คูแรง
Fx =10;
Fy =10;
Fz =10;
F = [Fx; Fy; Fz];
t = linspace(0, 6, 300);
```

รูปที่ 9 parameter ที่กำหนดในการ simulation

5.3 การคำนวณ Inverse Kinematics

ในการทำ inverse kinematic จะทำการคำนวณเพื่อทำ home configuration และ ใช้ในการจำลองการเคลื่อนที่ทั้งหมด DX,DY,DZ คือตำแหน่งของ EE ตลอดช่วงเวลาที่กำหนดเอาไว้

```

DZ = ((New_dz -dz)*t/6)+dz;
DX = ((New_dx -dx)*t/6)+dx;
DY = ((New_dy -dy)*t/6)+dy;

```

รูปที่ 10 แสดงสมการคำนวณเส้นทางการเคลื่อนที่ของตัวเลื่อน (DX, DY, DZ) จากตำแหน่งเริ่มต้นไปยังตำแหน่งปลายทาง

โดยการทำ inverse ในครั้งนี้จะทำการเก็บค่าออกมาให้องศาอยู่ฝั่งตามเข็มนาฬิกาซึ่งจะตรงกับการใช้งาน forward kinematic ที่ได้คำนวณเอาไว้ในส่วนของแกน X ที่ DX ต้องลบ 5 มาจากแกนแนว X ของ Tripteron จะไม่ต้องอยู่ที่ Y = 0 แต่ Y = 5 แทนไม้นั้น mechanic ของตัวหุ่นจะเกิดการขัดกันจากแขนแต่ละแขนชนกัน

```

for i = 1:length(t)

    q2x(i) = Invertq2(L1,L2,L3,DZ(i),DY(i),0);
    q1x(i) = Invertq1(L1,L2,L3,DZ(i),DY(i),q2x(i),0);
    q3x(i) = Invertq3(q1x(i),q2x(i),0);
    thetaX1 = q1x;
    thetaX2 = q2x;
    thetaX3 = q3x;

    q2y(i) = Invertq2(L1,L2,L3,DX(i),DZ(i),0);
    q1y(i) = Invertq1(L1,L2,L3,DX(i),DZ(i),q2y(i),0);
    q3y(i) = Invertq3(q1y(i),q2y(i),0);
    thetaY1 = q1y;
    thetaY2 = q2y;
    thetaY3 = q3y;

    q2z(i) = Invertq2(L1,L2,L3,-(5-DX(i)),-DY(i),180);
    q1z(i) = Invertq1(L1,L2,L3,-(5-DX(i)),-DY(i),q2z(i),180);
    q3z(i) = Invertq3(q1z(i),q2z(i),180);
    thetaZ1 = q1z;
    thetaZ2 = q2z;
    thetaZ3 = q3z;
end

```

รูปที่ 11 แสดงขั้นตอนการคำนวณ Inverse Kinematics ของทั้งสามขา X,Y,Z
ฟังก์ชันแต่ละตัวภายใน

1. ฟังก์ชันหา q_2

```
function q2 = Invertq2(L1, L2, L3, px, py, angle)
    angle_radian = deg2rad(angle);
    X = px - L3*cos(angle_radian);
    Y = py - L3*sin(angle_radian);
    cosB = ((X^2)+(Y^2)-(L1^2)-(L2^2))/(2*L1*L2);

    sinB = sqrt(1 - cosB^2);
    % sinB2 = -sqrt(1 - cosB^2);

    ck = atan2(sinB, cosB);
    if ck > 0 || ck < pi
        q2 = atan2(-sinB, cosB);
    else
        q2 = ck;
    end
    % r_2 = atan2(sinB2, cosB);

    % if r_1 > 1.5708 || r_1 < -1.5708
    %     q2 = r_2;
    % else if r_2 > 1.5708 || r_2 < -1.5708
    %     q2 = r_2;
    % end

end
```

รูปที่ 12 ฟังก์ชันหา q_2 โดยฟังก์ชันนี้รับข้อมูลตำแหน่งของปลายลิงก์ (px, py) ความยาวลิงก์ (L1, L2, L3) และมุมวางระนาบของขานั้น (angle)

จากนั้นคำนวณมุม q_2 โดยใช้กฎโคไซน์ (Law of Cosines)

2. ฟังก์ชันหา q_1

```
function q1 = Invertq1(L1, L2, L3, px, py, q1, angle)

    angle_radian = deg2rad(angle);
    X = px - L3*cos(angle_radian);
    Y = py - L3*sin(angle_radian);
    k1 = L1 + L2*cos(q1);
    k2 = L2*sin(q1);
    r = sqrt((k1^2)+(k2^2));
    cosAE = X / r;
    sinAE = Y / r;

    qAE = atan2(sinAE, cosAE);
    qE = atan2(k2, k1);
    q1 = qAE - qE;

end
```

รูปที่ 13 ฟังก์ชันหา q_1 โดยใช้ตำแหน่งปลายลิงก์ (px, py) พร้อมกับมุม q_2 ที่คำนวณได้จากฟังก์ชันก่อนหน้านี้ ฟังก์ชันใช้ความสัมพันธ์เชิงเรขาคณิตของโครงสร้างลิงก์ เพื่อหามุมระหว่างแนวแกนกับตำแหน่ง P และรีเทิร์นค่า q_1

3. ฟังก์ชันหา q_3

```
function q3 = Invertq3(q1,q2,angle)
    angle_radian = deg2rad(angle);
    ck = angle_radian - q1 - q2 ;
    if ck < - 2*pi
        q3 = ck + 2*pi;
    elseif (ck > 2*pi)
        q3 = ck - 2*pi;
    else
        q3 = ck;
    end
end
```

รูปที่ 14 ฟังก์ชันหา q_3 โดยฟังก์ชันนี้ นำมุมแพลนของขามาแปลงเป็นเรเดียน แล้วลบด้วยมุม q_1 และ q_2 เพื่อให้ได้มุมของข้อ 3 ตามสมการ

5.4 Forward kinematic

ขั้นตอนนี้จะทำการคำนวณหาตำแหน่งของแต่ละ joint ด้วยการทำ Forward kinematic โดยจะทำการผ่าน DH-convention จากนั้นจะเก็บค่าตำแหน่งเอาไว้ในตัวแปรเพื่อนำไปใช้ในการ simulate ต่อไป โดยขั้นตอนนี้จะพ่วงกับการ simulate ค่าที่เก็บไว้ตัวสุดท้ายจะเป็นตำแหน่ง joint สุดท้ายเพียงเท่านั้น

```

for k = 1:length(t)

%DH parameter
T0X = fowardx(0,DX(k),0, 0);
T01X = T0X * fowardx(0, 0,0, thetaX1(k)); % Joint 1 rotates X
T02X = T01X * fowardx(L1, 0,0, thetaX2(k));
T03X = T02X * fowardx(L2, 0, 0, thetaX3(k));
T0EX = T03X * fowardx(L3, 0, 0, 0);

T0Y = fowardy(0, DY(k),0, 0);
T01Y = T0Y * fowardy(0, 0,0, thetaY1(k)); % Joint 1 rotates y
T02Y = T01Y * fowardy(L1, 0,0, thetaY2(k));
T03Y = T02Y * fowardy(L2, 0, 0, thetaY3(k));
T0EY = T03Y * fowardy(L3, 0, 0, 0);

T0Z = fowardz(0, DZ(k),0, 0);
T01Z = T0Z * fowardz(0, 0,0, thetaZ1(k)); % Joint 1 rotates Z
T02Z = T01Z * fowardz(L1, 0,0, thetaZ2(k) );
T03Z = T02Z * fowardz(L2, 0, 0, thetaZ3(k) );
T0EZ = T03Z * fowardz(L3, 0, 0, 0);

%joint position

pX1 = T01X(1:3,4);
pX2 = T02X(1:3,4);
pX3 = T03X(1:3,4);
pX4 = T0EX(1:3,4);

pY1 = T0Y(1:3,4);
pY2 = T02Y(1:3,4);
pY3 = T03Y(1:3,4);
pY4 = T0EY(1:3,4);

pZ1 = T0Z(1:3,4);
pZ2 = T02Z(1:3,4);
pZ3 = T03Z(1:3,4);
pZ4 = T0EZ(1:3,4);
pZ1(1)=pZ1(1)+5;
pZ2(1)=pZ2(1)+5;
pZ3(1)=pZ3(1)+5;
pZ4(1)=pZ4(1)+5;

```

รูปที่ 15 แสดงลูปคำนวณ Forward Kinematics ของแต่ละขา
ฟังก์ชันแต่ละตัวภายใน

1. ฟังก์ชัน DH-convention แกน X

```

function T = fowardx(a, d, alpha, theta)
A = [1,0,0,0;
0,1,0,0;
0,0,1,a;
0,0,0,1];
Alpha = [cos(alpha),sin(alpha),0,0;
-sin(alpha),cos(alpha),0,0;
0,0,1,0;
0,0,0,1];
D = [1,0,0,d;
0,1,0,0;
0,0,1,0;
0,0,0,1];
Theta = [1,0,0,0;
0,cos(theta),sin(theta),0;
0,-sin(theta),cos(theta),0;
0,0,0,1];

T = A*Alpha*D*Theta;
end

```

รูปที่ 16 ฟังก์ชัน DH-Convention สำหรับแกน X

2. ฟังก์ชัน DH-convention แกน Y

```

function T = fowardy(a, d, alpha, theta)
    A = [1,0,0,a;
          0,1,0,0;
          0,0,1,0;
          0,0,0,1];
    Alpha = [1,0,0,0;
              0,cos(alpha),-sin(alpha),0;
              0,sin(alpha),cos(alpha),0;
              0,0,0,1];
    D = [1,0,0,0;
          0,1,0,d;
          0,0,1,0;
          0,0,0,1];
    Theta = [cos(theta),0,-sin(theta),0;
              0,1,0,0;
              sin(theta),0,cos(theta),0;
              0,0,0,1];

    T = A*Alpha*D*Theta;
end

```

รูปที่ 17 รูปที่ 16 ฟังก์ชัน DH-Convention สำหรับแกน Y

3. ฟังก์ชัน DH-convention แกน Z

```

function T = fowardz(a, d, alpha, theta)
    A = [1,0,0,a;
          0,1,0,0;
          0,0,1,0;
          0,0,0,1];
    Alpha = [1,0,-sin(alpha),0;
              0,cos(alpha),-sin(alpha),0;
              0,sin(alpha),cos(alpha),0;
              0,0,0,1];
    D = [1,0,0,0;
          0,1,0,0;
          0,0,1,d;
          0,0,0,1];
    Theta = [cos(theta),sin(theta),0,0;
              -sin(theta),cos(theta),0,0;
              0,0,1,0;
              0,0,0,1];

    T = A*Alpha*D*Theta;
end

```

รูปที่ 18 รูปที่ 16 ฟังก์ชัน DH-Convention สำหรับแกน Z

5.5 การจำลองการเคลื่อนที่ของ Tripteron

ในการทำ simulate จะสามารถเปิดปิดได้จากการเปลี่ยนค่าตัวแปร “simulate” ให้เป็น 0 กำหนด variable ของแผนภาพก่อน plot

- 1.กำหนดให้พื้นหลังเป็นสีขาว
- 2.กำหนดแกนหน้าหลังให้เป็น Z แกนซ้ายขวาให้เป็น X และ แกนขึ้นลงให้เป็น Y
- 3.กำหนดขนาดที่จะเห็นทั้งหมดภายในกราฟ
- 4.กำหนดมุมที่จะมอง

```

if simulate == 1
    figure('Color','white');
    hold on; grid on; axis equal;
    xlabel('Z'); ylabel('X'); zlabel('Y');
    title('tripteron Robot Simulation');

    xlim([-2*(sqrt(2)) 6]); ylim([-2*(sqrt(2)) 6+2*(sqrt(2))]); zlim([-2*(sqrt(2)) 6]);

    view(90,0); % 3D camera angle

    trace = animatedline('Color','m','LineWidth',1.5);
end

```

รูปที่ 19 รูปนี้แสดงโค้ดเริ่มต้นของการทำ Simulation ใน MATLAB โดยกำหนดสีพื้นหลัง, กำหนดแกน X-Y, ตั้งค่า view และ สร้างเส้น trace เพื่อแสดงการเคลื่อนที่ของ End-Effector ระหว่างการจำลอง

ขั้นตอนการ plot

- 1.กำหนด workspace ที่ Tripteron สามารถเคลื่อนที่ได้ทั้งหมดเป็นสีดำ
- 2.plot เส้นระหว่าง joint (joint 1 ไป joint 2) โดยที่ลำดับในการใส่ positionจะเป็นดังนี้ Z,X และ Y
- 3.แกน X ใช้สีแดง แกน Y ใช้สีเขียว แกน Z ใช้สีน้ำเงิน และ แกนที่เชื่อมต่อกับ EE จะใช้สีเหลือง

```
if simulate == 1
    cla;
    % Draw robot links (thick 3D lines)
    plot3([0 0], [0 0], [0 5], 'k', 'LineWidth',2);
    plot3([0 0], [0 5], [0 0], 'k', 'LineWidth',2);
    plot3([0 5], [5 5], [0 0], 'k', 'LineWidth',2);

    plot3([pX1(3) pX2(3)], [pX1(1) pX2(1)], [pX1(2) pX2(2)], 'r', 'LineWidth',2);
    plot3([pX2(3) pX3(3)], [pX2(1) pX3(1)], [pX2(2) pX3(2)], 'r', 'LineWidth',2);
    plot3([pX3(3) pX4(3)], [pX3(1) pX4(1)], [pX3(2) pX4(2)], 'y', 'LineWidth',4);
    %
    %
    plot3([pY1(3) pY2(3)], [pY1(1) pY2(1)], [pY1(2) pY2(2)], 'g', 'LineWidth',2);
    plot3([pY2(3) pY3(3)], [pY2(1) pY3(1)], [pY2(2) pY3(2)], 'g', 'LineWidth',2);
    plot3([pY3(3) pY4(3)], [pY3(1) pY4(1)], [pY3(2) pY4(2)], 'y', 'LineWidth',4);
    %
    %
    plot3([pZ1(3) pZ2(3)], [pZ1(1) pZ2(1)], [pZ1(2) pZ2(2)], 'b', 'LineWidth',2);
    plot3([pZ2(3) pZ3(3)], [pZ2(1) pZ3(1)], [pZ2(2) pZ3(2)], 'b', 'LineWidth',2);
    plot3([pZ3(3) pZ4(3)], [pZ3(1) pZ4(1)], [pZ3(2) pZ4(2)], 'y', 'LineWidth',4);

    % plot3([pZ3(1) pY3(1)], [pZ3(2) pY3(2)], [pZ3(3) pY3(3)], 'y', 'LineWidth',4);
    % plot3([pX3(1) pY3(1)], [pX3(2) pY3(2)], [pX3(3) pY3(3)], 'y', 'LineWidth',4);

    % Draw joints as spheres
```

รูปที่ 20 รูปนี้แสดงโค้ดที่ใช้วาดลิงก์ของ Tripteron ทั้ง 3 ขา (X, Y, Z)

- 4.plot แต่ละจุดเพื่อความสะดวกในการดู joint
- 5.กำหนดให้ทุกครั้งที่ loop จะเกิดการหมุนเท่าไร

```
% Draw joints as spheres
scatter3(pX1(3), pX1(1), pX1(2), 80, 'filled','r');
scatter3(pX2(3), pX2(1), pX2(2), 80, 'filled','r');
scatter3(pX3(3), pX3(1), pX3(2), 80, 'filled','r');
scatter3(pX4(3), pX4(1), pX4(2), 120, 'filled', 'y');
scatter3(pY1(3), pY1(1), pY1(2), 80, 'filled','g');
scatter3(pY2(3), pY2(1), pY2(2), 80, 'filled','g');
scatter3(pY3(3), pY3(1), pY3(2), 80, 'filled','g');
scatter3(pY4(3), pY4(1), pY4(2), 120, 'filled', 'y');
scatter3(pZ1(3), pZ1(1), pZ1(2), 80, 'filled','b');
scatter3(pZ2(3), pZ2(1), pZ2(2), 80, 'filled','b');
scatter3(pZ3(3), pZ3(1), pZ3(2), 80, 'filled','b');
scatter3(pZ4(3), pZ4(1), pZ4(2), 120, 'filled', 'y');

if ro == 1
    camorbit(0.3, 0);
end
drawnow limitrate;

%
end
end
```

รูปที่ 21 ภาพนี้แสดงโค้ดที่ใช้ plot จุด Joint ของ Tripteron ในแต่ละรอบของ loop

5.6 การคำนวณ Jacobian และ Static Force

หลังจากได้แบบจำลอง kinematics ของขาแต่ละขา (X-leg, Y-leg, Z-leg) ในรูปของสมการ Forward Kinematics แล้ว ผู้จัดทำได้นำสมการตำแหน่งปลายลิงก์ของโครงสร้าง Planar 3R มาคำนวณหาอนุพันธ์ตามมุมข้อต่อแต่ละข้อ เพื่อหาค่า Jacobian ของขาแต่ละขาในเชิงวิเคราะห์ (Analytical Jacobian) แล้วรวมให้อยู่ในรูปเมทริกซ์

$$J \in \mathbb{R}^{4 \times 3} \quad (61)$$

นั่นจึงนำสมการ Jacobian ที่ได้มาเขียนเป็นฟังก์ชันใน MATLAB ซึ่งรับพารามิเตอร์เป็นความยาวลิงก์ L1 L2 L3 และมุมข้อต่อ q_1, q_2, q_3 ของแต่ละขา แล้วคืนค่าตัว Jacobian ของขานั้น ๆ เพื่อนำไปใช้ในขั้นตอนการคำนวณ Static Force

```
function J = jacobian_X_full(L1,L2,L3,q1,q2,q3)

S1 = sin(q1); C1 = cos(q1);
S12 = sin(q1+q2); C12 = cos(q1+q2);
S123 = sin(q1+q2+q3);
C123 = cos(q1+q2+q3);

J = [
    0, 0, 0, 1;
    -L1*S1 - L2*S12 - L3*S123, -L2*S12 - L3*S123, -L3*S123, 0;
    L1*C1 + L2*C12 + L3*C123, L2*C12 + L3*C123, L3*C123, 0;
];
end
```

รูปที่ 22 ฟังก์ชันการคำนวณ Jacobian ขา X

```
function J = jacobian_Y_full(L1,L2,L3,q1,q2,q3)

S1 = sin(q1); C1 = cos(q1);
S12 = sin(q1+q2); C12 = cos(q1+q2);
S123 = sin(q1+q2+q3);
C123 = cos(q1+q2+q3);

J = [
    -L1*S1 - L2*S12 - L3*S123, -L2*S12 - L3*S123, -L3*S123, 0;
    0, 0, 0, 1;
    L1*C1 + L2*C12 + L3*C123, L2*C12 + L3*C123, L3*C123, 0;
];
end
```

รูปที่ 23 ฟังก์ชันการคำนวณ Jacobian ขา Y

```
function J = jacobian_Z_full(L1,L2,L3,q1,q2,q3)

S1 = sin(q1); C1 = cos(q1);
S12 = sin(q1+q2); C12 = cos(q1+q2);
S123 = sin(q1+q2+q3);
C123 = cos(q1+q2+q3);

J = [
    -L1*S1 - L2*S12 - L3*S123, -L2*S12 - L3*S123, -L3*S123, 0;
    L1*C1 + L2*C12 + L3*C123, L2*C12 + L3*C123, L3*C123, 0;
    0, 0, 0, 1;
];
end
```

รูปที่ 24 ฟังก์ชันการคำนวณ Jacobian ขา Z

ในการวิเคราะห์แรงสถิต ที่กำหนดแรงไว้จุด End-effector $F = \begin{bmatrix} F_x \\ F_y \\ F_z \end{bmatrix}$ โดยเราจะนำค่า Jacobian

ของแต่ละขาคำนวณตามสูตร $\tau = J^T F$ ทำให้ได้แรงที่ข้อต่อของแต่ละขา

$$\tau = [\tau_P, \tau_{R1}, \tau_{R2}, \tau_{R3}]^T$$

```
Jz_full = jacobian_Z_full(L1,L2,L3,thetaZ1(k),thetaZ2(k),thetaZ3(k));
Jx_full = jacobian_X_full(L1,L2,L3,thetaX1(k),thetaX2(k),thetaX3(k));
Jy_full = jacobian_Y_full(L1,L2,L3,thetaY1(k),thetaY2(k),thetaY3(k));

TauZ(:,k) = Jz_full' * F;
TauX(:,k) = Jx_full' * F;
TauY(:,k) = Jy_full' * F;
```

รูปที่ 25 การเรียกใช้งาน ฟังก์ชันคำนวณ Jacobian และการคำนวณหา Static torque

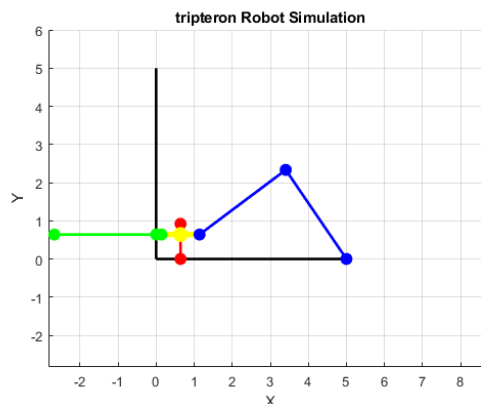
5.7 การทดสอบและแสดงผล

ภายใน MATLAB การจำลองนี้ได้กำหนดให้ End-Effector เคลื่อนที่จากตำแหน่งเริ่มต้น (0.5,0.5,0.5) ไป (4,4) และได้กำหนดแรงที่จุด EE โดย $F_z = 10$ และทำการสร้างกรอบกำหนดเวลา ตั้งแต่ $t = 0$ ถึง $t = 6$ จำนวน 300 จุด (Double Array) เพื่อจำลองการเคลื่อนที่ของ End-Effector จากตำแหน่งเริ่มต้นไปยังตำแหน่งเป้าหมาย โดยในแต่ละช่วงเวลา

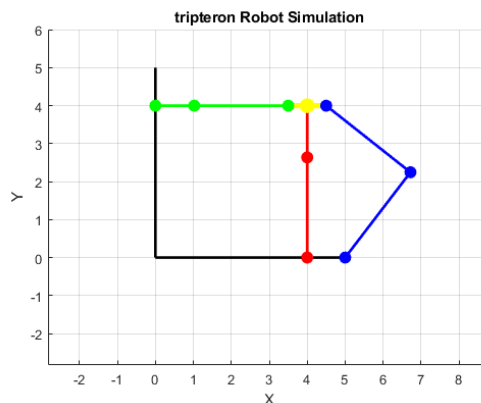
$$\Delta t = \frac{6}{300} = 0.02s \quad (62)$$

ค่าความละเอียดระดับนี้เพียงพอสำหรับการจำลองการเคลื่อนที่ที่เป็นไปอย่างต่อเนื่องของระบบ Tripteron

ทำให้เกิดภาพเคลื่อนไหว (Animation) ที่ลื่นไหล และทำให้ผลลัพธ์ของ Jacobian และ Torque มีความเสถียร

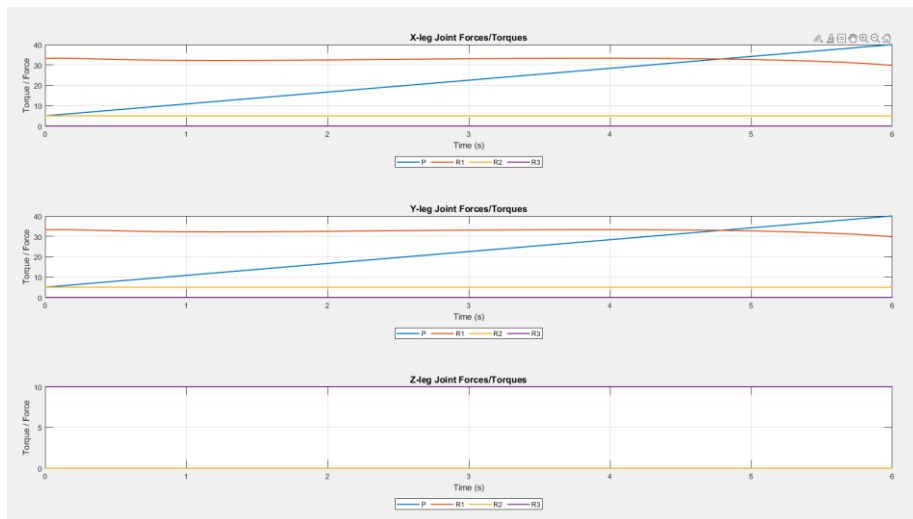


รูปที่ 26 ภาพการ *simulation* เมื่อ $t = 0$ สีแดง = X-leg สีเขียว = Y-leg สีฟ้า = Z-leg สีเหลือง = จุด EE



รูปที่ 27 ภาพการ *simulation* เมื่อ $t = 6$ สีแดง = X-leg สีเขียว = Y-leg สีฟ้า = Z-leg สีเหลือง = จุด EE

จะเห็นว่าโครงสร้างทั้งสามระนาบ (X-plane, Y-plane, Z-plane) ทำงานประสานกันเพื่อเลื่อน *EE* ไปยังตำแหน่งใหม่อย่างถูกต้องโดยไม่มีการชนกันของลิงก์



รูปที่ 28 การพล็อตกราฟ ของ *Different Kinematic* เพื่อดูแรงที่เกิดขึ้นในแต่ละข้อต่อ

กราฟทั้งสามแสดงค่าของแรงและแรงบิด (Torque) ที่เกิดขึ้นในขาแต่ละขาระหว่าง Simulation เมื่อมีแรง $F_z = 10$ กระทำที่ End-Effector ขา X และ Y รับภาระแรงส่วนใหญ่เนื่องจากเป็นโครงสร้างที่ช่วยพยุงการเคลื่อนที่ตามแกน Z จึงเกิดแรงบิดใน R1 และ R2 ชัดเจน ขา Z มีค่านิ่ง เพราะการกระทำของแรง F_z ส่งผลตรงกับแกนของ prismatic joint

```

===== TRIPTERON REPORT =====

===== ROBOT PARAMETERS =====
L1 = 2.8284    m    (Link 1 length)
L2 = 2.8284    m    (Link 2 length)
L3 = 0.5      m    (End-effector link offset)

===== INPUT FORCE / WRENCH PARAMETERS =====
Fx = 0      N
Fy = 0      N
Fz = 10     N

===== INPUT ACTUATOR POSITION (START → END) =====
DX = 0.5 → 4    m
DY = 0.5 → 4    m
DZ = 0.5 → 4    m

===== EE POSITION CHANGE (ACTUAL) =====
EE X: 0.5 → 4    m
EE Y: 0.5 → 4    m
EE Z: 0.5 → 4    m

===== VELOCITY RESULTS =====
--- EE VELOCITY (avg) ---
v_EE_x = 0.0019509    m/s
v_EE_z = 0.0019509    m/s
v_EE_y = 0.0019509    m/s
v_EE_total = 0.0033791    m/s

--- ACTUATOR SPEED (avg) ---
Actuator X = 0.58333    m/s
Actuator Y = 0.58333    m/s
Actuator Z = 0.58333    m/s

===== STATIC TORQUE ([Fx Fy Fz]) =====
--- X-leg (Joint0 = P, Joint1-3 = R) ---
τX0 (P) = 40    N
τX1 (R1) = 29.7868    N·m
τX2 (R2) = 5    N·m
τX3 (R3) = 0    N·m
--- Y-leg (Joint0 = P, Joint1-3 = R) ---
τY0 (P) = 40    N
τY1 (R1) = 29.7868    N·m
τY2 (R2) = 5    N·m
τY3 (R3) = 0    N·m
--- Z-leg (Joint0 = P, Joint1-3 = R) ---
τZ0 (P) = 0    N
τZ1 (R1) = 0    N·m
τZ2 (R2) = 0    N·m
τZ3 (R3) = 10    N·m

```

รูปที่ 29 แสดงผลการคำนวณสมรรถนะของระบบ Tripton ในโปรแกรม Matlab

ส่วนรายงานนี้นำเสนอค่าที่ได้จากการคำนวณ Jacobian และ Static Torque ที่จุดสุดท้ายของการเคลื่อนที่ (จุดที่ 300) โดยมีรายละเอียดดังนี้ ความเร็วเฉลี่ยของ End-Effector ในทั้ง 3 แกน ความเร็วของ prismatic actuator และแรงบิดที่เกิดขึ้นในข้อต่อ P-R1-R2-R3 ของแต่ละขา

การจำลองแสดงให้เห็นว่า Tripterion สามารถควบคุมตำแหน่ง End-Effector ได้อย่างเสถียรใน workspace ที่กำหนด การคำนวณ Jacobian และ Static Torque ให้ค่าที่สอดคล้องกับการเคลื่อนที่จริงของลิงก์

6. รายละเอียดโครงการ

ลำดับ	การดำเนินงาน	สัปดาห์ที่ 1	สัปดาห์ที่ 2	สัปดาห์ที่ 3	สัปดาห์ที่ 4	สัปดาห์ที่ 5	สัปดาห์ที่ 6
1	ศึกษาหาข้อมูลเกี่ยวกับ closed loop Kinematic Chain						
2	จัดทำ Proposal						
3	คำนวณ forward kinematic ของ Tripteron						
4	คำนวณ inverse kinematic ของ Tripteron						
5	คำนวณ different kinematic ของ Tripteron						
6	คำนวณ static force ของ Tripteron						
7	ทำ simulate ผ่าน MATLAB						
8	ปรับจูนแก้ไข และส่งงาน						

7. การแบ่งหน้าที่

ชื่อ	หน้าที่หลัก
นนทพงศ์ เสถียรพงศ์	คำนวณสมการเพื่อนำไปใช้ต่อใน MATLAB
วรภัทร ภัทรเปรมเจริญ	Simulate ผ่าน โปรแกรม MATLAB

8. เอกสารอ้างอิง (References)

- [1] YUANZHE ZHAO , POSITION AND FORCE ANALYSIS OF A 3-DOF PARALLEL TRANSLATIONAL MECHANISM , UNIVERSITY OF FLORIDA 2020
- [2] MECHATRONICS DESIGN AND KINEMATIC SIMULATION OF A TRIPTERON CARTESIAN-PARALLEL AGRICULTURAL ROBOT MOUNTED ON 4-WHEELED MOBILE PLATFORM TO PERFORM SEED SOWING ACTIVITY
FORWARD KINEMATIC
- [3] PATEL, Y. D., & GEORGE, P. M. (2012). *PARALLEL MANIPULATORS APPLICATIONS—A SURVEY*. INTERNATIONAL JOURNAL OF ENGINEERING RESEARCH AND APPLICATIONS
- [4] THE MATHWORKS, INC., "SIMULATE DYNAMIC SYSTEM BEHAVIOR - SIMULINK,"

[HTTPS://WWW.MATHWORKS.COM/HELP/SIMULINK/SIMULATION.HTML](https://www.mathworks.com/help/simulink/simulation.html)